

The 3-Stage "DMC-Intelligence" Pipeline

Stage 1: The "DMC-Gate" (Selective Detection & Instant Reject)

- **Model:** YOLOv11m (medium).
- **Role:** The spatial filter, trap-identifier, and instant-reject gate.
- **Input:** Raw 512\times512 image.
- **Logic:** The model scans for the 4 trained classes (Ground, Cage, Elevated, Trap Object).
 - If the model detects **0 authorized bins**, or if the only objects detected are **"Trap Objects"** (e.g., red postboxes, posters, Nilkamal bins), it instantly aborts further processing and **returns 0**.
 - If an authorized bin is found, it extracts the bounding box and passes the coordinates to Stage 2.
- **Crucial Constraint:** The model must be explicitly initialized with `device='cpu'` in the `predict.py` script to prevent crashes in the evaluation environment.

Stage 2: The "Contextual Observer" (Spatial Reasoning)

- **Model:** HTD-ViT (Hierarchical Token Decomposition Vision Transformer).
- **Role:** The core decision-maker assessing spills and overflows.
- **Input:** A cropped image of the detected authorized bin with a 50% spatial padding (the "Halo") to include the surrounding ground context.
- **Logic:** The ViT analyzes the semantic textures within the halo crop. It distinguishes between manmade garbage (plastic/paper) and natural litter (dry leaves), verifying if the garbage constitutes an active spill or overflow.
- **Output:** It acts as a strict binary classifier, outputting a **1** (Action Required) or **0** (No Action) for *each individual bin* it processes.

Stage 3: The "Deterministic Aggregation Head" (Final Output)

- **Model:** Lightweight Python Boolean Logic (Replacing the MSW-Net Stacking Strategy).
- **Role:** Resolving multi-bin images and generating the final strict integer output.
- **Input:** The array of binary states (0s and 1s) generated by Stage 2.
- **Logic:** Instead of probabilistic calibration, this stage applies a strict **OR** gate to handle edge cases where multiple bins are in a single frame:
 - If **ANY** bin in the image is flagged as **1** (spill/overflow) by the ViT, the final output is **1**.
 - If **ALL** bins in the image are flagged as **0** (clean/no spill), the final output is **0**.
- **Output:** A single integer (**0** or **1**) perfectly formatted for the evaluation scoring function.

Dataset Generation Pipeline

This document outlines the exact, step-by-step procedure for generating a highly robust, dual-stage dataset for the Signal Sprint competition. This pipeline is designed to maximize YOLO detection accuracy (Stage 1) while perfectly structuring the spatial-reasoning dataset for the HTD-VIT (Stage 2).

Phase 1: Capture & Dump (The Raw Master Set)

Goal: Collect a comprehensive, real-world dataset representing the true campus environment while maintaining strict constraints on "Trap Objects".

- **Camera Settings:** Ensure your phone camera is set to a 1:1 (Square) aspect ratio to prevent resolution distortion.
- **Transfer Method:** Use a local network tool (e.g., LocalSend) to wirelessly dump photos directly to your Ubuntu machine.
- **Target Directory:** `~/SignalSprint/ml_backend/data/raw_images/`
Total Photos Needed: ~2,000 Images.
- **The Capture Matrix:**
 - **450x Type 1 (Ground Bins):** Blue/Green/Cyan. Ensure a mix of clean surroundings, active spills/overflows, and bins surrounded only by natural litter.
 - **450x Type 2 (Metal Cage):** Ensure a mix of fully packed (clean exterior) and active spills outside the cage.
 - **450x Type 3 (Elevated Stand):** Blue/Green/Cyan. Ensure a mix of clean ground and active spills underneath.
 - **500x Trap Objects (Crucial):** Red postboxes, water drums, and 2D posters of bins.
 - **150x Pure Backgrounds (Crucial):** Empty ground, random scattered garbage with NO bins in sight, or pure natural leaves.

Phase 2: Roboflow Upload & Preprocessing

Goal: Prepare the raw images for annotation and standardizing the resolution to match the competition test set.

1. **Upload:** Create an Object Detection project in Roboflow and upload all 2,000 images from your `raw_images/` folder.
2. **Preprocess:** Add the Resize step.
3. **Settings:** Select Fit (Black Edges) and set the resolution to 512x512.

Phase 3: The Strict 4-Class Labeling & Mandatory State Tagging (Stage 1 Prep)

Goal: Define the bounding boxes for the YOLOv11 detector without confusing it with "state" conditions.

1. **The Bounding Boxes:** Draw tight bounding boxes around the bins and trap objects using exactly these 4 classes: `bin_ground`, `bin_cage`, `bin_elevated`, `trap_object` (Postboxes, Nilkamal, etc.).
2. **The "Null" SOTA Trick:** For the ~150 "Pure Background" images containing scattered garbage without bins, do not draw any boxes. Mark them as "Null". This teaches YOLO to reject garbage when an authorized bin is absent.
3. **Mandatory CSV State Tagging:** You must apply an image-level tag of either `action` or `no_action` to every annotated image to enable Phase 6 automation.
 - *Rule 1:* Only tag `action` if the spill is physically touching or directly underneath the bin (within its 50% halo).
 - *Rule 2:* Do not overcomplicate multi-bin images here; tag the overall state, and the Phase 6 script will handle the edge cases safely.

Phase 4: Augmentation & YOLO Export (Stage 1 Dataset)

Goal: Expand the dataset robustness against occlusion and lighting variations specifically for the YOLO training loop.

1. **Augmentations:** Apply the following settings in Roboflow: Horizontal Flip (50%) , Brightness (-20% to +20%).
2. **Generation:** Generate a 3x version of the dataset (Resulting in ~6,000 augmented images).
3. **Export:** Select "Export Dataset" in YOLOv11 PyTorch format.
4. **Target Directory:** Extract the downloaded files to:
`~/SignalSprint/ml_backend/data/yolo_dataset/`

Phase 5: The "Clean Export" & Automated Cropping (Stage 2 Prep)

Goal: Extract standardized "halo" crops around the authorized bins to feed the HTD-VIT classifier.

1. **Clean Export (Critical):** In Roboflow, generate a new version of your dataset with 0 augmentations (only the 512x512 resize). Export this to a temporary folder: `~/SignalSprint/ml_backend/data/temp_clean_export/`.
`+1`
2. **Run Extraction Script:** Execute your custom Python cropping script (`generate_vit_crops.py`). Logic: The script reads the bounding boxes, ignores `trap_object` classes, calculates a 50% spatial expansion (the

"halo"), and crops the images.

+1

3. **Target Directory:** The script will save ~1,350 unaugmented crops directly to:
~/SignalSprint/ml_backend/data/vit_dataset/unordered_crops
/

Phase 6: The Smart Automated Sort (Stage 2 Dataset Construction)

Goal: Define the binary classification (`action_required` vs. `no_action`) for the HTD-VIT model based strictly on DMC rules.

Run your custom Python sorting script utilizing the "Human-in-the-Loop" fallback logic:

1. **Read the Mapping:** The script parses the label files/CSV from the Phase 5 Clean Export to map filenames to your mandatory `action / no_action` tags.
2. **Automate Single Bins (90%):** If an image generated only one crop, the script trusts the CSV and automatically moves the crop to
~/vit_dataset/action_required/ or ~/vit_dataset/no_action/.
+1
3. **Human Verification for Multi-Bins (10%):** To prevent "Multi-Bin Poisoning," the script ignores the CSV if an image generated 2 or more crops. It pauses and uses OpenCV (`cv2.imshow`) to present the crops one by one on your screen. You manually press '1' (Action) or '0' (No Action) to properly route these edge cases.